

Seatbelt Detection System

Design, Approach & Justification Report

This report explains the design thinking behind a real-time seatbelt detection system built for in-vehicle cameras. It focuses on **why** each approach was chosen — the problem it solves, the trade-off it makes, and how it fits into the bigger picture. Two detection pipelines are presented, each suited to a different camera setup, and every major design decision is justified in plain terms.

Target device	Raspberry Pi 4 (in-vehicle, no internet)
Task	Detect whether the driver is wearing a seatbelt
Output	Real-time ON / OFF label overlaid on video
Key constraint	Must run fast enough to be useful in a moving vehicle

Contents

1	The Problem and Why It Is Hard	2
2	The Core Idea: Look at the Right Region First	2
3	Pipeline A: Direct Detection Approach	3
3.1	The Idea	3
3.2	Why YOLOv5 for Detection?	3
3.3	Why a Separate Classifier on Top?	3
3.4	Why Majority Voting for Smoothing?	4
3.5	Limitations of Pipeline A	4
4	Pipeline B: Body-Aware Approach	4
4.1	The Idea	4
4.2	Why Pose Estimation for the ROI?	5
4.3	Why Run Both YOLO and the CNN on the Cropped Region?	5
4.4	The Geometric Verification Step	5
4.5	Why the EMA Smoother Instead of Majority Vote?	6
4.6	The Optional BiLSTM: Learning Temporal Patterns	6
5	The Shared Classifier: Why One Model for Both Pipelines	7
6	Data Augmentation: Teaching the System About Real Conditions	7
7	Class Weighting: Accounting for Dataset Imbalance	8
8	Speed Optimisations: Making It Work on Embedded Hardware	9
8.1	Reduced Image Resolution	9
8.2	Frame Skipping	9
8.3	Background Video Capture	9
8.4	ONNX Runtime Export	9
8.5	Gated Geometric Check	9
9	Choosing Between the Two Pipelines	10
10	Why This Architecture Works as a Whole	10
11	Conclusion	11

1. The Problem and Why It Is Hard

At first glance, detecting a seatbelt seems straightforward — a diagonal strap across the chest is visually distinctive. In practice, however, several factors make this deceptively difficult.

The seatbelt is small and context-dependent. In a typical in-vehicle camera frame, the seatbelt strap occupies a narrow diagonal region of the image. Its colour blends with clothing, its shape changes with posture, and the relevant area shifts depending on where the camera is mounted.

Cameras are not standardised. A dashcam mounted at the top of the windscreen looks down at a steep angle. A camera in the dashboard looks forward. A rear-view mirror camera may be angled sideways. What the belt looks like in the image changes dramatically with camera position, and a system that works perfectly for one setup may completely fail for another.

Real-world conditions are messy. The strap is regularly obscured by the steering wheel, the driver's arm, a jacket, or a bag. Lighting changes from bright sun to underground car parks within minutes. Rapid head or body movements create motion blur.

Speed is non-negotiable. A Raspberry Pi 4 is a low-cost, power-efficient device well suited to in-vehicle deployment — but it has no dedicated graphics chip. Every component of the system must be lean enough to keep up with video in real time.

These four challenges shape every design decision in this project.

2. The Core Idea: Look at the Right Region First

The most important insight in this system is that **you should not search the whole image for a seatbelt**. If you ask a classifier to scan every pixel of a full-frame image, most of what it processes is irrelevant — the dashboard, the headrest, the road through the windscreen. This wastes computation and introduces noise.

A better approach is to **first identify where the relevant region is**, then run the detailed classification only on that cropped area. This two-step strategy — locate, then classify — is the backbone of both pipelines.

The two pipelines differ in *how* they find the relevant region. That difference is the central design choice of the system.

Central Design Principle

Find the torso first. Classify the seatbelt second. This keeps the classifier focused, fast, and accurate.

3. Pipeline A: Direct Detection Approach

3.1 The Idea

Pipeline A takes the most direct route. It uses a dedicated object detector — specifically a YOLOv5 model — that has been trained to find seatbelt straps in a full image. When the detector fires and draws a bounding box around the strap, that box is passed to a classifier that confirms whether it represents a worn seatbelt or not.

This works well because it mirrors the natural visual logic: spot the strap, then confirm what it is.

3.2 Why YOLOv5 for Detection?

YOLO (You Only Look Once) is a family of detectors that process an image in a single forward pass rather than proposing regions and then scoring each one separately. This makes it substantially faster than older detector architectures, which matters when every millisecond counts on a Raspberry Pi.

YOLOv5 specifically was chosen because:

- It is compact. The model used here has roughly 1.8 million parameters and runs at about 35 milliseconds per frame on the Raspberry Pi, which is fast enough for real-time use.
- It has extensive community support and proven reliability on embedded hardware.
- Pretrained weights from the COCO dataset give it a strong starting point before fine-tuning on seatbelt data.

3.3 Why a Separate Classifier on Top?

A natural question is: if the detector already finds the seatbelt, why run a second model at all? The detector answers “is there a strap here?” but not “is the driver actually wearing it?” A bag strap, a shoulder seam, or a seatbelt that has been pulled out but not buckled could all trigger the detector. The classifier — trained specifically on cropped torso patches labelled ON or OFF — provides a second opinion focused purely on the classification question. This separation of concerns makes each component simpler and more accurate.

3.4 Why Majority Voting for Smoothing?

Video is not a sequence of independent photos. The same driver, in the same seat, holds the same seatbelt state for seconds at a time. Yet individual frames can produce wrong predictions — due to a hand briefly covering the strap, a momentary blur, or unusual lighting. A majority vote over the last five frames corrects these transient errors without adding significant latency.

Five frames was chosen as a balance: short enough that the system reacts within a fraction of a second to a genuine state change, long enough to absorb a couple of bad frames in a row.

Why Pipeline A is the Faster Choice

Every frame passes through just two models: the YOLO detector and the CNN classifier. There is no pose estimation step. This makes Pipeline A faster — roughly 25 frames per second on the Raspberry Pi 4 — and ideal when the camera has a clear, direct view of the driver.

3.5 Limitations of Pipeline A

When Pipeline A Struggles

- **If the detector misses the strap entirely, the classifier is never called.** The system produces no output rather than a wrong output — which is safe, but unhelpful for angled cameras where the strap is not clearly visible.
- **No body context.** Without knowing where the shoulders and hips are, the system cannot distinguish a seatbelt strap from a similar-looking strap on a bag sitting in the passenger seat.
- **Majority voting can flip rapidly** on frames where the strap is borderline visible, producing an unstable label.

4. Pipeline B: Body-Aware Approach

4.1 The Idea

Pipeline B takes a different starting point. Instead of asking “where is the seatbelt strap?”, it first asks “where is the person’s body?” It uses a pose estimation model to find the driver’s shoulders and hips, crops the torso region, and only then runs both the YOLO detector and the CNN classifier on that cropped region. The two model outputs are then blended and smoothed over time.

This body-first approach is more robust to camera angles, occlusions, and driver movement, because the torso is much easier to find reliably than the narrow seatbelt strap.

4.2 Why Pose Estimation for the ROI?

MediaPipe Pose is a lightweight model developed by Google that detects body landmarks — specific joints like shoulders, elbows, and hips — from a single image. It is not trying to find the seatbelt; it is finding the person. This matters for two reasons.

First, the human torso is a large, consistent shape. Even when the seatbelt is hidden under a jacket or behind an arm, the shoulders and hips are almost always visible. This means the region of interest can be reliably located even in difficult frames.

Second, knowing exactly where the body is allows the crop to be tightly centred on the chest area — the region where the seatbelt should cross — rather than a loose, imprecise box around an uncertain strap location.

The system uses the *lite* variant of the MediaPipe model, which takes about 10 milliseconds per frame on the Raspberry Pi. The full-accuracy variant would take around 30 milliseconds, which is too costly for real-time use. The lite model sacrifices some precision on extreme body poses but is entirely adequate for standard driver postures.

4.3 Why Run Both YOLO and the CNN on the Cropped Region?

Running two models on the same crop might seem redundant, but they capture complementary information. The YOLO model looks for the seatbelt strap as a distinct object within the region. The CNN classifier looks at the overall texture and visual pattern of the torso to judge whether it looks like a person wearing or not wearing a seatbelt. When they agree, confidence is high. When they disagree, the system has a meaningful signal that the current frame is ambiguous and additional caution is warranted.

4.4 The Geometric Verification Step

A real seatbelt strap always crosses the torso diagonally, from the right shoulder down to the left hip (or vice versa). This is a physical constraint that pure neural networks do not explicitly encode. Pipeline B exploits this constraint as a tiebreaker.

When the YOLO and CNN models disagree — or when both give uncertain answers — the system runs a geometric verification step. It looks at the edge contours in the torso image and checks whether they form a diagonal line consistent with a real seatbelt trajectory. This check is purely based on geometry, not learned features, which means it catches cases where the neural networks are confused by texture or lighting but the shape of the strap is still present.

Importantly, this step is *gated* — it only runs when needed. If both the YOLO and CNN models already agree with high confidence, the geometric check is skipped entirely to

save time. This keeps the average latency low while providing a fallback for uncertain cases.

Why Geometric Verification Is a Smart Addition

Neural networks are excellent at recognising patterns they have seen in training but can be fooled by unusual textures, colours, or lighting. A belt is always a diagonal line. Combining a learned detector with a rule-based geometry check gives the system two independent ways to be correct, and two independent ways to catch errors.

4.5 Why the EMA Smoother Instead of Majority Vote?

Pipeline B uses an Exponential Moving Average (EMA) smoother rather than the majority vote used in Pipeline A. The difference is subtle but meaningful.

Majority voting treats all recent frames equally. A frame from five seconds ago has the same weight as the frame you just processed. The EMA gives more weight to recent frames and less weight to older ones, so the system naturally adapts to changes as they happen. It also produces a continuous confidence value rather than a binary vote count, which makes the final decision more nuanced.

The EMA has two tunable parameters. The smoothing factor controls how quickly the system reacts to changes: a lower value makes it more stable but slower to update, while a higher value makes it more responsive but noisier. The threshold controls how confident the EMA-smoothed probability needs to be before a positive (ON) label is declared.

Both parameters were tuned to lean slightly conservative — the system requires a bit more evidence to declare ON than to declare OFF, which is the right bias for a safety application. A missed seatbelt is more dangerous than a brief false alarm.

The system also introduces a *warm-up* period of six frames at the start of any video. During warm-up, the smoother labels the output as “WARMING” rather than committing to ON or OFF. This prevents unreliable early frames — before the EMA has accumulated enough history — from producing a confident but wrong decision.

4.6 The Optional BiLSTM: Learning Temporal Patterns

Beyond the EMA, an optional third stage introduces a bidirectional LSTM network. Unlike the EMA, which applies a fixed mathematical formula, the LSTM *learns* from examples how seatbelt states evolve over time. It can recognise, for instance, that a brief drop in confidence followed by a recovery is more likely a hand temporarily covering the belt than a genuine change in state.

This module requires labelled video sequences for training, which makes it optional — many deployments will not have that data available. When it is available, the LSTM

can provide smoother and more accurate temporal predictions than the EMA. When it is not, the EMA handles smoothing effectively on its own.

Why Pipeline B Is the More Robust Choice

By grounding its analysis in the driver’s body position first, Pipeline B remains reliable across a much wider range of camera angles and occlusion scenarios. The cost is a modest increase in processing time — roughly 5 milliseconds more per frame than Pipeline A — which is a worthwhile trade-off in challenging installation environments.

5. The Shared Classifier: Why One Model for Both Pipelines

Both pipelines use the same classifier: MobileNetV3-Small. This is a deliberate design choice that deserves explanation.

MobileNetV3-Small was designed specifically for deployment on devices with limited computing power. It is roughly four times smaller than a standard classification network but achieves comparable accuracy on most vision tasks. On a Raspberry Pi 4, it processes a single crop in about 5 milliseconds.

The model is not trained from scratch. It starts with weights learned on ImageNet, a dataset of over a million labelled photographs, which gives it a strong understanding of general visual features like edges, textures, and shapes. Only the final decision layer is replaced to output the two classes (ON and OFF), and the whole model is then fine-tuned on the seatbelt-specific patch dataset.

This approach — starting from a pretrained model and adapting it — is called transfer learning. It dramatically reduces the amount of labelled seatbelt data needed to achieve good performance, and it prevents the model from “forgetting” useful visual knowledge while learning the specific task.

The classifier takes 96x96 pixel crops as input. This size was chosen because it is small enough for fast processing but large enough to preserve the visual details that distinguish a worn seatbelt from a bare chest or a different strap.

6. Data Augmentation: Teaching the System About Real Conditions

Training a model on a fixed dataset risks teaching it only the conditions present in that dataset. If the training images were all taken in good lighting with a clear view, the

model may fail in dim car parks or when the driver’s arm is in the way. Augmentation addresses this by artificially creating variations of the training images.

Each of the augmentations chosen has a specific real-world motivation:

Augmentation	What real condition it simulates
Random crop, shift, rotate	Different camera mounting angles and driver postures
Coarse dropout (random patches)	A hand, arm, or steering wheel partially covering the belt
Brightness and contrast variation	Night driving, underground car parks, direct sunlight
Hue and saturation shift	Different seatbelt colours (black, grey, beige) and fabric materials
Motion blur	Camera shake, fast head turns, bumpy roads
Gaussian noise	Low-quality camera sensors and digital compression

Crucially, no augmentation is applied to validation images. The validation set is always processed with plain resizing and normalisation, so that evaluation scores reflect genuine generalisation and not artificially boosted performance.

7. Class Weighting: Accounting for Dataset Imbalance

In most driving scenarios, the seatbelt is being worn. This means that a dataset collected from real footage will typically have significantly more ON images than OFF images. If you train a classifier on an imbalanced dataset without correction, it tends to learn that simply predicting the majority class (ON) most of the time gets it a decent accuracy score, even though it is badly wrong for the more important minority class (OFF).

This system addresses this with class weighting. During training, each sample’s contribution to the loss is multiplied by a weight inversely proportional to how common that class is. Rare OFF samples are treated as more important, which forces the model to learn to distinguish them properly rather than ignoring them.

This is especially important for a safety-critical system. False negatives — failing to detect that the seatbelt is not worn — are more dangerous than false positives, and the class weighting pushes the model in the right direction.

8. Speed Optimisations: Making It Work on Embedded Hardware

Getting accurate detection is only half the challenge. Getting it to run fast enough on a Raspberry Pi 4 to be useful in a moving vehicle is the other half. Several deliberate decisions were made to achieve this.

8.1 Reduced Image Resolution

The YOLO detector was trained and runs at 320x320 pixels rather than the more common 640x640. This reduces the amount of computation by roughly 75% with only a minor hit to detection accuracy. For close-range, in-vehicle use, the seatbelt strap is large enough relative to the image that a lower resolution is entirely adequate.

8.2 Frame Skipping

Not every frame needs to be fully processed. In a smooth video at 30 frames per second, adjacent frames are nearly identical. The system can skip processing every other frame and simply carry forward the result from the previous frame. This halves the computational load with almost no perceptible effect on accuracy, since seatbelt state does not change faster than the eye can see.

8.3 Background Video Capture

Reading frames from the camera and processing them happen on separate threads. This prevents the system from stalling while waiting for the next frame to arrive, keeping the processor busy with inference instead of idle waiting. It is a small but meaningful throughput improvement.

8.4 ONNX Runtime Export

The trained PyTorch models can be exported to the ONNX format and run through ONNX Runtime — a dedicated inference engine optimised for CPU execution. On the Raspberry Pi 4, this delivers roughly 3 additional frames per second over running the model in its original PyTorch form, purely by using more efficient CPU operations.

8.5 Gated Geometric Check

As noted earlier, the RANSAC geometric verification step only runs when the neural network models disagree or are uncertain. For frames where both models give confident, consistent answers, this step is entirely skipped. This means the extra cost of geometry checking is only paid when it is genuinely needed.

9. Choosing Between the Two Pipelines

The two pipelines are not in competition — they solve different problems. The choice between them depends entirely on the physical camera setup.

Decision Guide

Use Pipeline A when: the camera is mounted directly in front of the driver (dashboard or steering column), the seatbelt strap is consistently visible in the image, and maximum speed is a priority. Pipeline A runs at approximately 25 frames per second on the Raspberry Pi 4.

Use Pipeline B when: the camera is mounted at an angle (high on the windscreen, on the rear-view mirror, or side-facing), the strap is frequently partially occluded, or the deployment involves multiple different vehicles with different camera positions. Pipeline B runs at approximately 20 frames per second on the Raspberry Pi 4.

The five-frames-per-second difference between the pipelines is unlikely to matter in practice. Both are fast enough for smooth real-time overlay at the target frame rate. The accuracy and robustness difference, however, can be significant in difficult camera conditions.

Table 1: Practical comparison of the two pipelines across real-world scenarios.

Scenario	Pipeline A	Pipeline B
Camera directly faces driver	Works excellently	Also works well
Camera mounted at an angle	May miss the strap	Body-first approach handles it
Driver’s arm covers the belt	Low confidence or miss	CNN still reads the torso
Unusual seatbelt colour	Depends on training data	Same, but torso context helps
Multiple people in frame	Detects each independently	Tracks one person at a time
Empty seat	No output (no strap detected)	Falls back to full frame

10. Why This Architecture Works as a Whole

Stepping back from the individual components, the system embodies a layered defensive strategy. At each stage, a different type of evidence is gathered, and each type catches failures that the others miss.

The neural network detector (YOLO) is excellent at recognising the distinctive shape of a seatbelt strap when it is clearly visible. The CNN classifier adds a texture-level confirmation that looks at the overall appearance of the torso region, catching cases where the strap is visible but unusual. The geometric check adds a physics-based constraint that neither neural network explicitly encodes. The temporal smoother ensures that a single bad frame does not cause an alarm, while still reacting quickly to genuine changes.

No single one of these components is perfect. But because their failure modes are different, the combination is substantially more reliable than any one of them alone. This principle — layering complementary evidence sources so that errors in one layer are caught by another — is the key reason the system can be trusted in the unpredictable conditions of real driving.

“The question is never whether a single model can be fooled. In real conditions, every model can be fooled by something. The question is whether the system as a whole can still reach the right answer when one of its components is confused. That is what layering is for.”

11. Conclusion

This system demonstrates that effective, real-time seatbelt detection on low-cost embedded hardware is achievable through thoughtful design choices rather than raw computational power.

The key decisions — and the reasons behind them — are:

1. **Locate before classifying.** Always crop to the relevant region before running the classifier. This keeps inference fast and accurate.
2. **Two location strategies for two camera realities.** YOLOv5 for direct-view cameras (Pipeline A), MediaPipe pose estimation for angled cameras (Pipeline B). The right strategy depends on the installation.
3. **Multiple evidence sources.** Combining an object detector, a texture classifier, and a geometry check gives the system three independent ways to reach the correct answer and three independent ways to catch errors.
4. **Temporal smoothing is not optional.** Video frames are not independent. Ignoring time leads to unstable, flickering predictions. The EMA smoother makes the output consistent and trustworthy.
5. **Speed through restraint.** Reducing image resolution, skipping frames, and ex-

porting to ONNX Runtime are not compromises — they are deliberate choices that make the system deployable on the target hardware.

6. **Safety bias.** The training loss, the confidence threshold, and the decision logic all lean toward caution. In a safety application, it is always better to raise a question than to silently miss a problem.

Together, these choices produce a system that is fast enough to run in real time, robust enough to handle the variability of real driving conditions, and honest enough about its own uncertainty to be trusted in a safety-critical context.
